

# FinanzasMMEX

## Plan de Arquitectura — App Desktop de Consolidación Financiera Local

*Sincronización offline-first de Gmail, scraping headful, Mercado Pago API y PDFs hacia finanza.mmb  
(Money Manager Ex)*

Autor: Ricardo • Asistencia técnica: Claude (Anthropic)

Versión 1.0 • 2 de mayo de 2026

Síntesis de cuatro informes técnicos (Claude, Gemini, NotebookLM, OpenAI) + skill spec-driven-workflow

# Tabla de contenidos

*La tabla se actualiza con clic-derecho → "Actualizar campos" en Word, o F9 en LibreOffice.*

|                                                  |    |
|--------------------------------------------------|----|
| Tabla de contenidos.....                         | 2  |
| 0. Resumen ejecutivo.....                        | 6  |
| Decisión arquitectónica central .....            | 6  |
| Estrategia de fuentes (chilena, mayo 2026) ..... | 6  |
| Stack técnico final .....                        | 7  |
| Roadmap de entrega.....                          | 7  |
| 1. Especificación funcional (SPEC).....          | 9  |
| 1.1 Propósito .....                              | 9  |
| 1.2 Contexto y personas .....                    | 9  |
| 1.3 Requisitos funcionales .....                 | 9  |
| Ingesta.....                                     | 9  |
| Normalización y categorización .....             | 10 |
| Deduplicación.....                               | 10 |
| Conciliación .....                               | 10 |
| Integración con MMEX.....                        | 10 |
| Multi-usuario y privacidad .....                 | 10 |
| 1.4 Requisitos no funcionales .....              | 11 |
| 1.5 Inputs y outputs .....                       | 11 |
| Inputs .....                                     | 11 |
| Outputs .....                                    | 12 |
| 1.6 Criterios de éxito .....                     | 12 |
| 1.7 Fuera de alcance (versión 1) .....           | 12 |
| 1.8 Restricciones técnicas .....                 | 13 |
| 1.9 Marco legal chileno aplicable.....           | 13 |
| 2. Arquitectura del sistema.....                 | 15 |
| 2.1 Visión a 30,000 pies .....                   | 15 |
| 2.2 Arquitectura de dos capas .....              | 15 |
| Por qué dos capas y no una .....                 | 15 |
| 2.3 Componentes y responsabilidades .....        | 16 |
| Inventario de módulos.....                       | 16 |
| Contrato de interfaz central: CanonicalTx .....  | 17 |
| 2.4 Modelo de datos canónico (staging.db) .....  | 18 |

|                                                           |    |
|-----------------------------------------------------------|----|
| Tabla principal: canonical_tx .....                       | 18 |
| Tabla de soporte: raw_artifacts .....                     | 19 |
| Tabla de reglas: category_rules .....                     | 19 |
| Tabla de auditoría: merge_log y reconcile_log.....        | 20 |
| 2.5 Mapeo a tablas MMEX.....                              | 20 |
| 2.6 Estrategia de fuentes (detalle por banco) .....       | 21 |
| BancoEstado.....                                          | 21 |
| Banco Falabella / CMR .....                               | 21 |
| MachBCI .....                                             | 22 |
| Mercado Pago .....                                        | 22 |
| 2.7 Pipelines de ingestión .....                          | 22 |
| Pipeline Gmail .....                                      | 22 |
| Pipeline Mercado Pago API .....                           | 23 |
| Pipeline Playwright headful (semi-asistido).....          | 23 |
| 2.8 Motor ETL: normalización + categorización.....        | 23 |
| Pipeline ETL .....                                        | 23 |
| Diccionario inicial de normalización .....                | 24 |
| LLM fallback (Ollama + Qwen3 8B) .....                    | 25 |
| 2.9 Estrategia de deduplicación.....                      | 25 |
| Cómo se calcula fitid_synthetic .....                     | 25 |
| Sliding window para conciliación email ↔ extracto.....    | 26 |
| Reglas de fusión cuando hay match.....                    | 26 |
| Detección de transferencias internas .....                | 26 |
| 2.10 Conciliación matemática ("Regla de Oro") .....       | 27 |
| 2.11 Estrategia multi-usuario.....                        | 27 |
| Cuentas separadas (no datos separados) .....              | 27 |
| 2.12 Gastos compartidos: Tags .....                       | 28 |
| 2.13 Estructura de archivos del proyecto .....            | 28 |
| 2.14 Concurrencia: el problema de MMEX abierto .....      | 29 |
| 3. Decisiones técnicas (con justificación).....           | 30 |
| 3.1 Python local + SQLite directo (vs CSV en MVP).....    | 30 |
| 3.2 OAuth Gmail API en lugar de IMAP.....                 | 30 |
| 3.3 Ollama/Qwen3 8B como fallback semántico.....          | 30 |
| 3.4 Playwright headful para BancoEstado y Falabella ..... | 30 |
| 3.5 Mercado Pago vía OAuth .....                          | 31 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 3.6 MachBCI: solo email + manual.....                        | 31 |
| 3.7 FITID sintético en CUSTOMFIELDDATA_V1 (no en NOTES)..... | 31 |
| 3.8 Tags vs Categorías para gastos conjuntos.....            | 31 |
| 3.9 Backups: cómo y cuándo.....                              | 31 |
| 3.10 Cifrado: .mmb vs .emb vs contenedor BitLocker.....      | 32 |
| 3.11 Refactoring del proyecto previo gmail-to-MMEX.....      | 32 |
| 4. Roadmap por fases .....                                   | 33 |
| Fase 0 — Setup (1–2 días).....                               | 33 |
| Fase 1 — MVP (3–5 días) .....                                | 33 |
| Fase 2 — Writer SQLite directo (3–4 días).....               | 34 |
| Fase 3 — Scraping headful (4–6 días) .....                   | 35 |
| Fase 4 — Extras (3–5 días) .....                             | 35 |
| Fase 5 — Onboarding Laura (variable).....                    | 35 |
| 5. Plan de pruebas.....                                      | 37 |
| 5.1 Estrategia general .....                                 | 37 |
| 5.2 Tests unitarios.....                                     | 37 |
| 5.3 Tests de integración .....                               | 37 |
| 5.4 Tests manuales (smoke tests).....                        | 37 |
| 5.5 Tests de seguridad .....                                 | 37 |
| 6. Operación y mantenimiento .....                           | 39 |
| 6.1 Logging.....                                             | 39 |
| 6.2 Cuándo intervenir manualmente.....                       | 39 |
| 6.3 Procedimiento de restauración .....                      | 39 |
| 6.4 Mantenimiento del diccionario de categorización .....    | 39 |
| 7. Riesgos y mitigaciones .....                              | 41 |
| Apéndice A — Origen de cada decisión por informe.....        | 43 |
| Decisiones donde el plan diverge de los informes .....       | 44 |
| Apéndice B — Esquema SQL completo de staging.db.....         | 45 |
| Apéndice C — Patrones regex iniciales (BancoEstado).....     | 48 |
| Compra con tarjeta de crédito.....                           | 48 |
| Transferencia recibida.....                                  | 48 |
| Transferencia enviada.....                                   | 48 |
| Pago automático de servicio.....                             | 48 |
| Retiro en cajero.....                                        | 48 |
| Helper de normalización de montos chilenos .....             | 49 |

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Apéndice D — Compliance chileno: detalle adicional .....                             | 50 |
| D.1 Posición legal de la operación .....                                             | 50 |
| D.2 Riesgo contractual con bancos .....                                              | 50 |
| D.3 Cronograma SFA (Sistema de Finanzas Abiertas) .....                              | 50 |
| Apéndice E — Plantilla de acuerdo de mandato .....                                   | 52 |
| ACUERDO DE MANDATO Y CONSENTIMIENTO INFORMADO PARA GESTIÓN FINANCIERA FAMILIAR ..... | 52 |
| Cierre y siguientes pasos.....                                                       | 54 |
| Próximos pasos sugeridos.....                                                        | 54 |
| Contacto y mantenimiento del documento.....                                          | 54 |

## 0. Resumen ejecutivo

Este documento es el plan de arquitectura para FinanzasMMEX: una aplicación de escritorio Python que consolida transacciones financieras de la familia (Ricardo + Laura) desde múltiples fuentes chilenas hacia un único archivo Money Manager Ex (finanza.mmb), operando 100% en la PC local de Ricardo, sin depender de servicios en la nube ni de agregadores comerciales como Fintoc o Floid.

El plan integra y reconcilia los hallazgos de cuatro informes técnicos previos: el reporte de Claude (con investigación de campo sobre el estado real del web banking chileno y restricciones legales 2026), el de Gemini (ETL clásico con Pandas y custom fields), el de NotebookLM (LLM local + fuzzy matching + tags) y el de OpenAI (modelo canónico de dos capas y formatos abiertos como OFX/MT940). Donde los informes discrepan, este documento toma decisiones explícitas y las justifica.

### Decisión arquitectónica central

La aplicación NO escribe directamente sobre finanza.mmb desde la primera versión. Adopta una arquitectura de dos capas:

- **Capa de staging** — una base SQLite separada (staging.db) con un esquema canónico rico, donde aterrizan crudos todos los datos extraídos. Aquí ocurre la normalización, deduplicación, conciliación matemática y categorización.
- **Capa de integración** — un componente que toma transacciones validadas en staging y las inyecta en finanza.mmb. En la Fase 1 lo hace exportando OFX o CSV e importando vía la UI de MMEX (delega validación al motor maduro de MMEX). En la Fase 2 lo hace por SQL directo con transacciones ACID.

Esta separación es la decisión más importante del plan. Permite reimportar, revertir y auditar sin riesgo de corromper el archivo MMEX, mantiene trazabilidad completa del origen de cada transacción y desacopla la lógica de extracción de los detalles del esquema MMEX.

### Estrategia de fuentes (chilena, mayo 2026)

El siguiente cuadro resume la estrategia primaria y de contingencia por fuente. Refleja el estado real del ecosistema chileno (no la teoría general):

| Fuente                | Pipeline primario                                              | Pipeline de contingencia           | Veredicto        |
|-----------------------|----------------------------------------------------------------|------------------------------------|------------------|
| BancoEstado           | Email Gmail (primaria) + scraping Playwright headful (semanal) | OFX export + manual cuando falle   | Email + scraping |
| Banco Falabella / CMR | Email Gmail (primaria)                                         | Scraping Playwright headful        | Email + scraping |
| MachBCI               | Email Gmail (cuando hay notificación)                          | Export manual mensual desde la app | Email + manual   |
| Mercado Pago          | API oficial OAuth (primaria, desatendida)                      | Email Gmail como respaldo          | API + email      |

| Fuente                          | Pipeline primario                | Pipeline de contingencia | Veredicto |
|---------------------------------|----------------------------------|--------------------------|-----------|
| Tarjetas presenciales sin email | Entrada manual (PWA o quick-add) | —                        | Manual    |

### Stack técnico final

- **Lenguaje:** Python 3.12 (alineado con su entorno actual y con el proyecto previo de gmail-to-MMEX).
- **Persistencia local:** SQLite (staging.db) y SQLite vía MMEX (finanza.mmb).
- **Email:** Google Gmail API con OAuth (refresh token persistido localmente). Se descarta IMAP por ser más frágil con el 2FA actual de Google.
- **Scraping web:** Playwright headful con storage\_state. Solo BancoEstado y Falabella, ejecución semi-asistida (Ricardo presente al iniciar).
- **PDFs:** pdfplumber para texto nativo + Camelot (lattice) para tablas con bordes + pytesseract+OCRmyPDF para escaneados.
- **Parsing semántico de emails:** Regex como capa primaria (rápida, determinista) + LLM local Ollama/Qwen3 8B como fallback cuando el regex falla.
- **Fuzzy matching:** RapidFuzz para normalizar comercios y emparejar transacciones de email vs. extracto.
- **API banca:** Mercado Pago Developer App (Authorization Code OAuth) — única API legítima del set.
- **Orquestación:** Windows Task Scheduler con tareas separadas por fuente.
- **UI auxiliar:** PySide6 (Qt) para una pequeña ventana de revisión / quick-add manual; reusable de su experiencia previa con PyQt6.
- **Tests:** pytest + fixtures de emails reales anonimizados.
- **Logs y observabilidad:** structlog → archivo rotatorio + opcional ingesta a Sentinel para auditoría posterior.

### Roadmap de entrega

- **Fase 0 (1–2 días):** setup, esqueleto de proyecto, staging.db, OAuth Gmail y MP.
- **Fase 1 — MVP (3–5 días):** ingesta Gmail + Mercado Pago API → staging.db → exportación OFX → import manual a MMEX.
- **Fase 2 (3–4 días):** writer SQLite directo a finanza.mmb con transacciones ACID y custom fields.
- **Fase 3 (4–6 días):** scraping headful para BancoEstado y Falabella.
- **Fase 4 (3–5 días):** PDFs de contingencia + dashboard + reportes.
- **Fase 5 (variable):** onboarding de Laura y gastos compartidos por tags.

**Recordatorio importante (legal Chile)**

El proyecto se sostiene en el principio de "acceso con credenciales propias" según la Ley 21.459 (delitos informáticos) art. 2: el acceso con tus propias credenciales y consentimiento explícito de Laura no es "indebido". El riesgo principal NO es penal sino contractual (T&C de BancoEstado y Falabella prohíben acceso automatizado). La mitigación es: scraping semi-asistido, no desatendido, con cadencia humana razonable y sin alteración de datos. La Norma CMF de Sistema de Finanzas Abiertas (NCG en vigor desde julio 2026) abrirá un canal API regulado en 12–24 meses; arquitecturar las fuentes como pluggable permitirá migrar a SFA sin reescribir el sistema.



# 1. Especificación funcional (SPEC)

Esta sección es la "single source of truth" del proyecto, siguiendo el skill spec-driven-workflow. Cuando algo en el código no coincida con la spec, se vuelve a este documento, no a parchear código.

## 1.1 Propósito

Mantener finanza.mmb (Money Manager Ex de Ricardo y Laura) actualizado de forma confiable, automática y privada, sin entregar credenciales bancarias a terceros, sin depender de servicios en la nube de pago, y sin requerir entrada manual diaria de transacciones.

La meta funcional es: al final de cada día, finanza.mmb refleja todas las transacciones del día anterior con categorización correcta, sin duplicados y con saldos cuadrados contra los balances reportados por los bancos.

## 1.2 Contexto y personas

| Actor   | Descripción                                                                                                                                                                                                                          |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ricardo | Usuario principal, técnico (IT/seguridad). Ejecuta el sistema, hace mantenimiento, revisa excepciones. Tiene cuentas en BancoEstado, Falabella/CMR, MachBCI y Mercado Pago. Trabaja en Windows 11.                                   |
| Laura   | Pareja de Ricardo. Usaria pasiva del sistema; sus transacciones también deben quedar en finanza.mmb. Usa BancoEstado/Falabella propias. Ricardo tiene su consentimiento explícito documentado para acceder con credenciales de ella. |
| Sistema | Corre como tareas programadas en la PC de Ricardo. Solo escribe en finanza.mmb cuando MMEX está cerrado. Si MMEX está abierto, escribe a staging.db y espera.                                                                        |

## 1.3 Requisitos funcionales

### Ingesta

- **FR-1:** El sistema debe leer correos de Gmail filtrados por el label "Bancos" y extraer transacciones de BancoEstado, Falabella, MachBCI y Mercado Pago.
- **FR-2:** El sistema debe consultar la API de Mercado Pago vía OAuth y obtener movimientos de cuenta usando /v1/account/settlement\_report y /v1/payments/search.
- **FR-3:** El sistema debe permitir scraping web headful (con Ricardo presente al iniciar la sesión) de BancoEstado y Falabella para obtener movimientos no presentes en email.
- **FR-4:** El sistema debe procesar archivos OFX, QIF, CSV y PDF dejados por Ricardo o Laura en una carpeta de "drop" local.

- **FR-5:** El sistema debe permitir entrada manual rápida de transacciones (compras presenciales sin email/SMS).

### Normalización y categorización

- **FR-6:** Toda transacción extraída debe ser normalizada al esquema canónico de staging.db antes de cualquier escritura a MMEX.
- **FR-7:** El sistema debe normalizar el nombre del comercio (merchant\_raw → merchant\_norm) usando un diccionario de reglas locales más fuzzy matching (RapidFuzz).
- **FR-8:** El sistema debe asignar PAYEEID y CATEGID automáticamente cuando el comercio normalizado coincida con uno previamente categorizado en MMEX.
- **FR-9:** Cuando el regex y el diccionario no logren parsear un email, el sistema debe invocar Ollama (Qwen3 8B) localmente como fallback semántico, con un prompt JSON-schema-strict.

### Deduplicación

- **FR-10:** El sistema debe generar un hash determinista (SHA-256) sobre (fecha, monto, cuenta\_o\_tarjeta, merchant\_norm) y usarlo como FITID sintético para evitar duplicados entre las fuentes email/scraping/OFX/manual.
- **FR-11:** El sistema debe aplicar una sliding window de  $\pm 14$  días para emparejar la notificación rápida de email (autorización) con su liquidación posterior en el extracto.
- **FR-12:** El hash sintético debe persistirse en CUSTOMFIELDDATA\_V1 de MMEX (campo personalizado dedicado), no en el campo NOTES.

### Conciliación

- **FR-13:** Para cada extracto bancario procesado, el sistema debe verificar matemáticamente que  $B_{final} = B_{inicial} + \Sigma \text{depósitos} - \Sigma \text{retiros}$  y marcar el lote como "reconciled" o "needs review".
- **FR-14:** El sistema debe identificar transferencias internas entre cuentas familiares modeladas en MMEX (retiro en cuenta A + depósito por mismo monto en cuenta B dentro de 24h) y representarlas como Transfer (TOACCOUNTID), no como ingreso + gasto separados.

### Integración con MMEX

- **FR-15:** El sistema debe respetar el archivo finanza.mmb: nunca escribir si MMEX está abierto (lock detection). En lugar de fallar, debe diferir la escritura a la próxima ventana en que MMEX esté cerrado.
- **FR-16:** Toda escritura a finanza.mmb debe estar envuelta en BEGIN IMMEDIATE / COMMIT / ROLLBACK con copia de respaldo previa.
- **FR-17:** El sistema debe poder operar en modo "export-only": generar OFX/CSV en una carpeta y dejar que Ricardo importe manualmente desde la UI de MMEX.

### Multi-usuario y privacidad

- **FR-18:** Cada cuenta y tarjeta debe tener su propio ACCOUNTID en MMEX ("Mi tarjeta", "Tarjeta Laura").
- **FR-19:** Los gastos compartidos deben usar TAG\_V1 ("Conjunto") sin duplicar la transacción.
- **FR-20:** Los datos sensibles (tokens OAuth, storage\_state de Playwright, contraseñas) deben almacenarse en Windows Credential Manager (vía keyring), nunca en archivos de texto plano del repo.

## 1.4 Requisitos no funcionales

- **NFR-1 (Privacidad):** ningún dato transaccional, credencial ni archivo bancario sale de la PC de Ricardo. La única conexión externa autorizada es Gmail API y Mercado Pago API.
- **NFR-2 (Seguridad):** todo dato en reposo (staging.db, finanza.mmb, OFX intermedios) reside en una carpeta protegida por BitLocker o equivalente. El propio finanza.mmb se mantiene en formato .mmb durante la operación; la conversión a .emb cifrado puede hacerse a discreción del usuario.
- **NFR-3 (Confiabilidad):** el sistema debe ser idempotente: ejecutar dos veces el mismo input no genera duplicados. La pérdida de transacciones es peor que el duplicado: ante duda, marca para revisión.
- **NFR-4 (Auditabilidad):** cada transacción en finanza.mmb debe poder ser trazada a un email\_id, OFX file, scraping run o entrada manual concreta vía staging.db.
- **NFR-5 (Resiliencia):** cambios en plantillas de correo bancario deben degradar gracefully (log + fallback LLM), no perder transacciones silenciosamente.
- **NFR-6 (Latencia):** la ingesta de un email nuevo debe procesarse en menos de 60s; la consulta a Mercado Pago, en menos de 30s.
- **NFR-7 (Reversibilidad):** todo estado debe ser reproducible desde fuentes originales (Gmail + carpeta de drop + manual ledger). Si finanza.mmb se corrompe, se restaura desde backup y se re-ejecuta el pipeline.
- **NFR-8 (Mantenibilidad):** código en Python idiomático, 100% type-hinted, formateado con ruff, con cobertura  $\geq 70\%$  en módulos de parsing y ETL.

## 1.5 Inputs y outputs

### Inputs

- Mensajes Gmail con label "Bancos" (lectura vía API).
- Llamadas autenticadas a Mercado Pago Developer API (OAuth refresh token persistido).
- Sesiones Playwright headful contra portales bancarios (storage\_state local).
- Archivos en C:\Finanzas\drop\ : OFX, QIF, CSV, PDF, XLSX exportados manualmente.
- Entrada manual via UI Qt o CSV manual\_entries.csv.

## Outputs

- Inserciones en finanza.mmb (CHECKINGACCOUNT\_V1, PAYEE\_V1, CATEGORY\_V1, TAG\_V1, TAGLINK\_V1, CUSTOMFIELDDATA\_V1).
- Archivo OFX intermedio (export por cuenta) que Ricardo puede importar manualmente como contingencia.
- Logs estructurados en C:\Finanzas\logs\.
- Reporte HTML local de "transacciones para revisión" generado al final de cada ejecución.

## 1.6 Criterios de éxito

- **SC-1:** Dada una bandeja Gmail con 30 días de notificaciones BancoEstado, el pipeline extrae  $\geq 95\%$  de las transacciones reales sin duplicados, en  $\leq 5$  minutos.
- **SC-2:** Dado un OFX exportado del portal de BancoEstado y los emails de los mismos 30 días, el sistema fusiona ambos sin generar duplicados (deduplicación por FITID sintético funciona).
- **SC-3:** Una transferencia entre cuenta de Ricardo y cuenta de Laura aparece en MMEX como Transfer único, no como egreso+ingreso.
- **SC-4:** Si MMEX está abierto al momento de ejecutar Task Scheduler, el writer detecta el lock y difiere; los datos quedan en staging.db y se aplican en la próxima ventana.
- **SC-5:** El balance final reportado por BancoEstado en su email/portal coincide al peso con la suma de transacciones importadas.
- **SC-6:** Tras ejecutar el pipeline dos veces consecutivas con los mismos inputs, no se crean nuevos registros en finanza.mmb (idempotencia).
- **SC-7:** Si BancoEstado cambia su plantilla de correo, las transacciones del día siguen entrando vía LLM fallback y un alert HTML lo señala.
- **SC-8:** Para una compra normalizada ("AMZN MKTP" → Amazon, Categoría: Compras), la categoría se aplica correctamente en  $> 90\%$  de los casos sin intervención humana.
- **SC-9:** Tiempo de adopción para Laura:  $< 1$  hora (ella solo necesita confirmar su consentimiento por escrito y no instalar nada en su PC).

## 1.7 Fuera de alcance (versión 1)

- Soporte para más de un archivo finanza.mmb simultáneo.
- Investing/portfolio (acciones, fondos): MMEX lo soporta pero no se carga aquí.
- Sincronización entre dispositivos (Syncthing, etc.): se trata como tarea futura, opcional.
- App móvil propia. La PWA/quick-add se integra como sub-tarea de Fase 5, no es bloqueante.
- Conciliación con tarjetas Visa/MasterCard "presenciales" sin email ni SMS — entran como manual.
- Ingesta de transacciones empresariales (Virtual IT Trust): es otro dominio, otro archivo MMEX si se necesita.

- Soporte para .emb cifrado nativo: la operación diaria se hace sobre .mmb plano dentro de un contenedor BitLocker. Si Ricardo decide migrar a .emb, requiere un módulo extra de descifrado/recifrado al inicio/fin de cada ejecución.

## 1.8 Restricciones técnicas

- **TC-1:** Windows 11 + Python 3.12 64-bit como entorno productivo principal.
- **TC-2:** Versión MMEX objetivo: 1.9.2 (enero 2026), schema v21. El sistema debe leer ACCOUNTLIST\_V1, CATEGORY\_V1.PARENTID (categorías anidadas desde 1.7) y soportar TAG\_V1+TAGLINK\_V1.
- **TC-3:** Hardware base: laptop ASUS ExpertBook 16GB RAM. Ollama corre Qwen3 8B en CPU (latencia aceptable de ~10s por prompt).
- **TC-4:** Sin dependencia de servicios cloud salvo Gmail API y Mercado Pago API. Cero dependencias hacia OpenAI, Anthropic o cualquier LLM remoto.
- **TC-5:** El sistema debe convivir con el proyecto previo de gmail-to-MMEX si existe; no rompe esa carpeta y se ofrece como reemplazo evolutivo.

## 1.9 Marco legal chileno aplicable

La operación se sostiene legalmente en tres normas vigentes en mayo 2026. Esta sección es la base para la respuesta a cualquier pregunta de compliance que Ricardo necesite documentar.

| Norma                             | Aplicación                                                                             | Implicancia para FinanzasMMEX                                                                                                                                                                                                                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ley 21.459 (Delitos Informáticos) | Castiga el "acceso indebido a un sistema informático" (art. 2)                         | <b>El acceso con credenciales propias y consentimiento expreso de Laura no es "indebido".</b> El riesgo penal es bajo. El riesgo real es contractual: los T&C de BancoEstado y Falabella prohíben acceso automatizado, lo que puede gatillar suspensión de cuenta. Mitigación: scraping semi-asistido con cadencia humana. |
| Ley 21.521 (Fintec) + NCG SFA     | El Sistema de Finanzas Abiertas entra en vigor 4 julio 2026 con implementación gradual | En 12–24 meses surgirá un canal API regulado para acceso de datos por terceros con consentimiento del usuario. <b>Recomendación arquitectónica: las fuentes deben ser pluggable.</b> Cuando SFA llegue a banca minorista, se reemplaza el módulo scraping_be.py por sfa_be.py sin tocar el resto del sistema.              |

| Norma                         | Aplicación                                                               | Implicancia para FinanzasMMEX                                                                                                                                                                                                                  |
|-------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ley 21.719 (Datos Personales) | Vigente desde diciembre 2026 — el titular tiene derechos sobre sus datos | Ricardo es titular de sus datos y de los de Laura (con consentimiento expreso documentado). <b>Mantener todo local con cifrado en reposo (BitLocker en la carpeta C:\Finanzas\)</b> es la práctica correcta. Ningún tercero procesa los datos. |

#### Documento que conviene generar (one-shot)

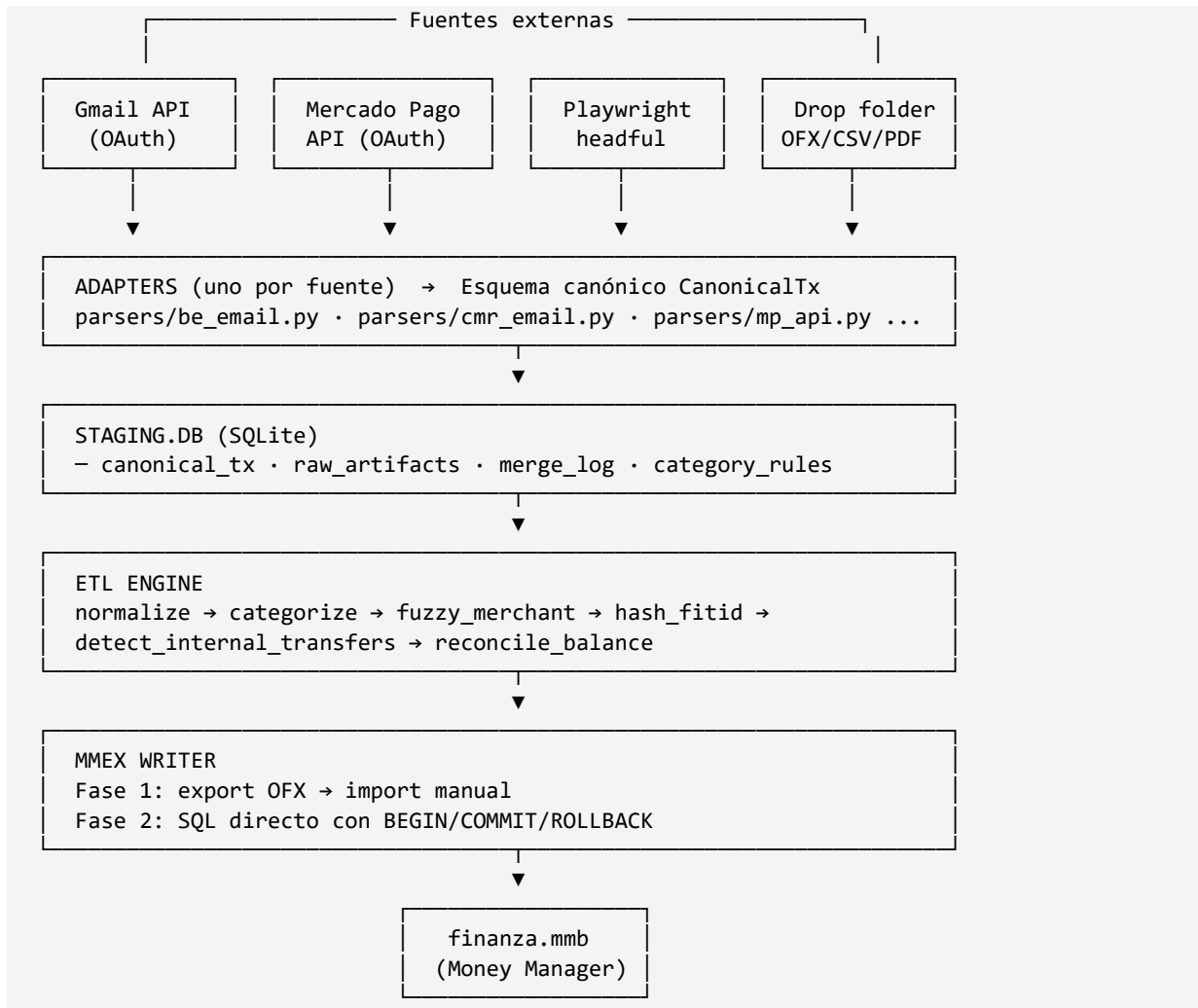
Antes del primer scraping de cuenta de Laura, Ricardo debe firmar con ella un "Acuerdo de mandato y consentimiento informado" que: (a) declare que Ricardo accede con credenciales que ella le entrega voluntariamente, (b) limite el alcance al portal bancario para fines de gestión financiera familiar, (c) sea revocable en cualquier momento por escrito. Una página, en español, archivada en PDF en la carpeta del proyecto. No es exigible por nadie hoy, pero es la mejor evidencia documental ante cualquier disputa futura.

## 2. Arquitectura del sistema

### 2.1 Visión a 30,000 pies

FinanzasMMEX se modela como un pipeline de tres etapas (Extract → Transform → Load) con persistencia intermedia en SQLite y dos colas: una de "transacciones nuevas para insertar" y otra de "transacciones que requieren revisión humana".

El flujo conceptual es el siguiente (representado como ASCII para que sobreviva en cualquier visor):



### 2.2 Arquitectura de dos capas

Esta es la decisión central, defendida por los cuatro informes con énfasis especialmente fuerte en el de OpenAI: separar la base de staging de la base productiva.

#### Por qué dos capas y no una

| Beneficio                    | Justificación                                                                                                                                                                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Atomicidad y rollback        | staging permite reintentar lotes completos sin riesgo. Si la inserción a finanza.mmb falla a mitad de camino, el rollback de SQLite restaura el estado y los datos siguen sanos en staging. |
| Idempotencia                 | el hash FITID sintético se calcula y persiste en staging antes de tocar MMEX. Volver a correr el pipeline con los mismos inputs no produce duplicados.                                      |
| Auditabilidad                | cada fila en finanza.mmb se vincula vía hash a su artefacto crudo (raw_text del email, OFX file path, scraping HTML snapshot).                                                              |
| Multi-fuente                 | una transacción de tarjeta puede aparecer en email Y en OFX semanas después; la fusión ocurre en staging, MMEX solo recibe el registro consolidado.                                         |
| Evolución                    | si un día se cambia de MMEX a Firefly III o a otra herramienta, el writer es lo único que se reescribe. Toda la inteligencia de extracción/normalización se preserva.                       |
| Convivencia con MMEX abierto | si Ricardo está editando finanza.mmb manualmente, staging puede seguir aceptando ingestas. El writer espera la próxima ventana de cierre.                                                   |

## 2.3 Componentes y responsabilidades

### Inventario de módulos

| Carpeta       | Responsabilidad                                                                                       | Archivos clave                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| adapters/     | Un parser por fuente. Convierte input crudo a CanonicalTx.                                            | be_email.py, cmr_email.py, mach_email.py, mp_api.py, scraping_be.py, scraping_cmr.py, ofx_loader.py, csv_loader.py, pdf_loader.py, manual_entry.py |
| staging/      | Capa de persistencia y queries sobre staging.db.                                                      | schema.sql, repo.py, migrations/                                                                                                                   |
| etl/          | Lógica de normalización, fuzzy match, hash FITID, detección de transferencias internas, conciliación. | normalize.py, fuzzy.py, fitid.py, transfers.py, reconcile.py, llm_fallback.py                                                                      |
| writer/       | Escritura a finanza.mmb (OFX export en Fase 1; SQL directo en Fase 2).                                | ofx_export.py, mmex_sql.py, mmex_lock.py, backup.py                                                                                                |
| orchestrator/ | Glue: orquesta cada fuente como un Job, gestiona excepciones y retries, escribe logs.                 | jobs.py, scheduler.py, alerts.py                                                                                                                   |
| ui/           | Pequeña UI Qt para review queue, quick-add manual y status del pipeline.                              | review_window.py, quickadd.py, status_panel.py                                                                                                     |
| secrets/      | Wrapper a Windows Credential Manager / keyring. Único punto de acceso a tokens y storage_state.       | vault.py                                                                                                                                           |



| Carpeta | Responsabilidad                                                                  | Archivos clave                                               |
|---------|----------------------------------------------------------------------------------|--------------------------------------------------------------|
| cli.py  | Entrypoint command-line: finanzasmex run --source gmail   mp   scraping-be   all |                                                              |
| tests/  | pytest. Fixtures con emails reales anonimizados.                                 | test_be_email.py, test_fitid.py, test_writer.py, test_e2e.py |

## Contrato de interfaz central: CanonicalTx

Todos los adapters producen instancias de la misma clase. Esto es el "protocol layer" del sistema.

```
@dataclass(frozen=True)
class CanonicalTx:
    # Identidad y origen
    tx_uid: str                # UUID4 generado al ingestar
    owner: Literal['ricardo', 'laura', 'joint']
    source_type: Literal['email', 'mp_api', 'scraping', 'ofx',
                        'csv', 'pdf', 'manual']
    source_file: str | None    # path local o gmail message-id
    source_ref: str | None     # message-id, run_id de scraping, etc.
    raw_text: str              # texto crudo del email/HTML/PDF
    content_sha256: str        # hash del raw_text completo

    # Temporalidad
    event_date: date | None    # fecha de la transacción real
    booking_date: date | None  # fecha contable del banco
    posted_date: date | None   # fecha en que aparece en el extracto

    # Cuantitativo
    amount: Decimal            # SIEMPRE positivo, dirección por separado
    currency: str               # CLP por default, USD si aplica
    direction: Literal['debit', 'credit']

    # Cuenta / tarjeta
    account_alias: str          # 'BE_Ricardo_RUT', 'CMR_Laura'
    card_last4: str | None

    # Comercio
    merchant_raw: str           # como vino del banco
    merchant_norm: str | None   # normalizado por dict + RapidFuzz

    # Tipología contable
    tx_type: Literal['purchase', 'bill_payment', 'transfer_in',
                    'transfer_out', 'internal_transfer',
                    'refund', 'cash_withdrawal', 'fee']

    # Categorización
    category_guess: str | None  # 'Comida', 'Servicios'
    subcategory_guess: str | None
    tags: list[str]             # ['joint'], ['personal']
```

```
# Trazabilidad
parser_name: str          # 'be_email_v3'
parser_version: str
needs_review: bool        # True si hay ambigüedad

# FITID sintético (calculado en ETL, no en adapter)
fitid_synthetic: str | None = None
```

Esta clase es inmutable (frozen=True). Cada modificación produce una copia nueva, lo que facilita el rastreo y los tests.

## 2.4 Modelo de datos canónico (staging.db)

staging.db es una base SQLite local que vive en C:\Finanzas\staging\staging.db. Su esquema es más rico que el de MMEX porque guarda metadatos de origen, hashes y audit trail.

### Tabla principal: canonical\_tx

```
CREATE TABLE canonical_tx (
  tx_uid          TEXT PRIMARY KEY,
  owner           TEXT NOT NULL CHECK (owner IN ('ricardo','laura','joint')),
  source_type     TEXT NOT NULL,
  source_file     TEXT,
  source_ref      TEXT,
  content_sha256  TEXT NOT NULL,
  raw_text        TEXT,

  event_date      TEXT,
  booking_date    TEXT,
  posted_date     TEXT,

  amount          REAL NOT NULL CHECK (amount > 0),
  currency        TEXT NOT NULL DEFAULT 'CLP',
  direction       TEXT NOT NULL CHECK (direction IN ('debit','credit')),

  account_alias   TEXT NOT NULL,
  card_last4      TEXT,

  merchant_raw    TEXT,
  merchant_norm   TEXT,

  tx_type         TEXT NOT NULL,
  category_guess  TEXT,
  subcategory_guess TEXT,
  tags_json       TEXT,          -- JSON array

  fitid_synthetic TEXT NOT NULL, -- hash determinista
  parser_name     TEXT NOT NULL,
  parser_version  TEXT NOT NULL,
  needs_review    INTEGER NOT NULL DEFAULT 0,
  review_reason   TEXT,
```

```

-- Estado en MMEX
mmex_account_id      INTEGER,
mmex_tx_id           INTEGER,      -- TRANSID en CHECKINGACCOUNT_V1
mmex_status          TEXT NOT NULL DEFAULT 'pending'
                    CHECK (mmex_status IN ('pending','exported','
                    inserted','rejected')),

created_at           TEXT NOT NULL DEFAULT (datetime('now')),
updated_at           TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE UNIQUE INDEX uq_canonical_fitid ON canonical_tx(fitid_synthetic);
CREATE INDEX idx_canonical_owner_date ON canonical_tx(owner, posted_date);
CREATE INDEX idx_canonical_status      ON canonical_tx(mmex_status);
CREATE INDEX idx_canonical_match      ON canonical_tx(
owner, account_alias, amount, posted_date, merchant_norm);

```

### Tabla de soporte: raw\_artifacts

Cada email, HTML scrapeado, PDF original, response JSON de Mercado Pago se guarda íntegro aquí. Es la "fuente de verdad" para reprocesar todo si el parser cambia.

```

CREATE TABLE raw_artifacts (
  artifact_id      TEXT PRIMARY KEY,      -- UUID4
  artifact_type    TEXT NOT NULL,         -- 'email','mp_response','html','pdf'
  source_ref       TEXT NOT NULL,         -- gmail msg-id, run_id, file path
  content_sha256   TEXT NOT NULL,
  payload_blob     BLOB,                  -- contenido binario crudo
  payload_text     TEXT,                  -- contenido textual extraído
  fetched_at       TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE UNIQUE INDEX uq_raw_sha ON raw_artifacts(content_sha256);

```

### Tabla de reglas: category\_rules

Diccionario local de normalización merchant → categoría/payee. Es editable por Ricardo a mano (sqlite browser). Se carga al inicio en memoria.

```

CREATE TABLE category_rules (
  rule_id          INTEGER PRIMARY KEY,
  pattern          TEXT NOT NULL,         -- substring o regex
  pattern_type     TEXT NOT NULL CHECK (pattern_type IN ('substr','regex','fuzzy')),
  merchant_norm    TEXT NOT NULL,         -- 'Amazon'
  payee_id_mmex    INTEGER,               -- PAYEEID en MMEX (si ya existe)
  category_name    TEXT NOT NULL,         -- 'Compras Online'
  subcategory_name TEXT,
  fuzzy_threshold  INTEGER DEFAULT 85,    -- 0-100, solo si type=fuzzy
  priority         INTEGER NOT NULL DEFAULT 100,
  active          INTEGER NOT NULL DEFAULT 1,
  created_at       TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE INDEX idx_rules_active_priority ON category_rules(active, priority);

```

## Tabla de auditoría: merge\_log y reconcile\_log

Cada fusión de transacciones (email + OFX correspondiente) y cada reconciliación de balance se registra. Es la base para el reporte HTML diario.

```
CREATE TABLE merge_log (
  merge_id      INTEGER PRIMARY KEY,
  fitid_synthetic TEXT NOT NULL,
  sources_json  TEXT NOT NULL,      -- ['email#abc', 'ofx#xyz']
  chosen_source TEXT NOT NULL,      -- cuál ganó cada campo
  merged_at     TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE reconcile_log (
  reconcile_id    INTEGER PRIMARY KEY,
  account_alias   TEXT NOT NULL,
  period_start    TEXT NOT NULL,
  period_end      TEXT NOT NULL,
  balance_initial REAL NOT NULL,
  balance_final   REAL NOT NULL,
  sum_credits     REAL NOT NULL,
  sum_debits     REAL NOT NULL,
  expected_final  REAL NOT NULL,
  status          TEXT NOT NULL CHECK (status IN ('ok','off','manual_review')),
  delta           REAL NOT NULL,    -- balance_final - expected_final
  notes          TEXT,
  created_at     TEXT NOT NULL DEFAULT (datetime('now'))
);
```

## 2.5 Mapeo a tablas MMEX

Money Manager Ex 1.9.2 (schema v21) expone su base SQLite con tablas que terminan en \_V1. Esta es la traducción de CanonicalTx → registros MMEX.

| Campo CanonicalTx | Destino en MMEX                                                                            | Notas                                                                                           |
|-------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| tx_uid            | (no se mapea — vive solo en staging)                                                       | Trazabilidad interna                                                                            |
| fitid_synthetic   | CUSTOMFIELDDATA_V1.CONTENT (REFID = TRANSID, FIELDID = ID del campo "Hash")                | DEDUPE central. Este es el truco para evitar el problema histórico de duplicados al reimportar. |
| account_alias     | ACCOUNTLIST_V1.ACCOUNTNAME → resolver a ACCOUNTID                                          | Cada alias del staging mapea a una cuenta única en MMEX.                                        |
| merchant_norm     | PAYEE_V1.PAYEENAME → resolver/crear PAYEEID                                                | Si el payee no existe, se crea con CATEGID=-1 (sin categoría por default).                      |
| category_guess    | CATEGORY_V1.CATEGNAME → CATEGID (con CATEGORY_V1.PARENTID para subcategorías anidadas v21) | Mantener jerarquía padre/hijo desde MMEX 1.7+.                                                  |

| Campo CanonicalTx      | Destino en MMEX                                                     | Notas                                                                                                                   |
|------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| amount + direction     | CHECKINGACCOUNT_V1.TRANSAMOUNT +<br>TRANSCODE                       | TRANSCODE='Withdrawal' si<br>direction='debit', 'Deposit' si<br>'credit', 'Transfer' si<br>tx_type='internal_transfer'. |
| posted_date            | CHECKINGACCOUNT_V1.TRANSDATE<br>(formato ISO 'YYYY-MM-DD')          | Validación contra fecha futura y<br>contra fecha < 2020-01-01.                                                          |
| raw_text               | ATTACHMENT_V1 (si > 4KB) o NOTES (si<br>corto)                      | Texto largo va como adjunto, no<br>como NOTES.                                                                          |
| tags (joint, personal) | TAG_V1 + TAGLINK_V1 (REFID=TRANSID,<br>TAGID resuelto desde TAG_V1) | El motor de reportes de MMEX<br>permite filtrar por tags.                                                               |
| needs_review = true    | CHECKINGACCOUNT_V1.STATUS = 'F'<br>(Followup)                       | Aprovecha el marcador nativo de<br>MMEX para revisión humana.                                                           |
| internal_transfer      | TRANSCODE='Transfer' + TOACCOUNTID +<br>TOTRANSAMOUNT               | Solo si ambos extremos son<br>cuentas modeladas en MMEX.                                                                |

#### Tabla CHECKINGACCOUNT\_V1 (esquema completo, MMEX 1.9.2 v21)

Las columnas relevantes para inserción son: TRANSID (PK), ACCOUNTID, TOACCOUNTID, PAYEEID, TRANSCODE ('Withdrawal'|'Deposit'|'Transfer'), TRANSAMOUNT, STATUS ('R'|'F'|'V'|'D'|'), TRANSACTIONNUMBER, NOTES, CATEGID, TRANSDATE, LASTUPDATEDTIME, FOLLOWUPID, TOTRANSAMOUNT, COLOR, DELETEDTIME. La columna FITID YA EXISTE en v21 (añadida por la utilidad OfxToMmex). Esto significa que si Ricardo importa OFX en lugar de hacer INSERT directo, MMEX hace deduplicación automática por FITID — y esa es la ruta menos invasiva para Fase 1.

## 2.6 Estrategia de fuentes (detalle por banco)

### BancoEstado

- **Estado real (mayo 2026):** el portal soporta lectura headful con BE Pass (push al celular una vez por sesión) y storage\_state válido por horas. Headless es bloqueado por TLS fingerprinting + dispositivo móvil registrado. La ventana de tarjeta de coordenadas se cerró completamente en agosto 2026.
- **Pipeline primario:** Gmail API con regex sobre asunto y cuerpo. Latencia ~1 minuto por transacción.
- **Pipeline contingencia:** Playwright headful, ejecución semanal con Ricardo presente al inicio. Solo lectura de movimientos. Export OFX cuando esté disponible.
- **Cuándo Ricardo necesita intervenir:** 1 sesión semanal de ~5 minutos para refrescar storage\_state.

### Banco Falabella / CMR

- **Estado real:** el sitio permite leer movimientos en modo headful, pero cualquier acción transaccional (y a veces el login mismo) gatilla Clave Dinámica vía SMS. Para SOLO LECTURA, el flujo semi-auto funciona bien.
- **Pipeline primario:** Gmail API (las notificaciones de compra CMR llegan por email para casi todo).
- **Pipeline contingencia:** Playwright headful con storage\_state. La sesión es estable mientras no expire.

## MachBCI

- **Estado real:** no tiene web banking público para personas. El historial está solo en la app móvil. La API en docs.business-api.soymach.com es exclusiva para comercios; no hay endpoint para movimientos personales.
- **Vías reales:** (a) emails de notificación (cuando estén configurados), (b) export manual mensual desde la app.
- **Decisión:** no se intenta scraping ni API privada. Mach queda en modo "email + manual". Es la fuente con menor cobertura automatizada y eso se acepta como restricción del entorno.

## Mercado Pago

- **Estado real:** tiene API oficial documentada (/v1/payments/search, /v1/account/settlement\_report/config) accesible con OAuth.
- **Pipeline primario:** API oficial. Es la única fuente con API legítima del set. Es el camino más limpio y el primero que debe atacarse.
- **Implementación:** crear una "aplicación" de developer en Mercado Pago, autorizar con la cuenta de Ricardo vía Authorization Code flow una sola vez, persistir refresh\_token en keyring. La tarea programada hace refresh y consulta sin intervención humana.

## 2.7 Pipelines de ingestión

### Pipeline Gmail

Esta es la fuente más rica en cobertura. Funciona desatendida vía Task Scheduler.

```
# orchestrator/jobs.py – pseudocódigo del job Gmail

def run_gmail_sync(since_iso: str = None) -> JobResult:
    creds = vault.get_oauth('gmail') # refresh automático
    service = build('gmail', 'v1', credentials=creds)

    query = f'label:Bancos newer_than:1d' if since_iso is None \
            else f'label:Bancos after:{since_iso}'

    msgs = list_messages(service, query)

    for msg in msgs:
        artifact = persist_raw_artifact(msg) # raw_artifacts table
```

```

sender    = extract_header(msg, 'From')
adapter   = ADAPTER_REGISTRY.resolve(sender) # be|cmr|mach|mp|other
if adapter is None:
    log.warning('unknown_sender', sender=sender)
    continue
try:
    tx_list = adapter.parse(msg, artifact) # → list[CanonicalTx]
except ParseError as e:
    tx_list = llm_fallback.parse(msg, artifact, hint=adapter.name)
for tx in tx_list:
    staging.upsert(tx) # idempotente por fitid

return JobResult(processed=len(msgs))

```

## Pipeline Mercado Pago API

Cadencia: 6 horas. Refresca token automáticamente.

```

# adapters/mp_api.py – pseudocódigo del job MP

def fetch_mp_movements(since: date) -> list[CanonicalTx]:
    token = vault.get_oauth('mp_api').refresh_if_expired()
    payments = mp_client.payments.search(
        token=token,
        begin_date=since.isoformat(),
        end_date=date.today().isoformat(),
        sort='date_created',
        order='asc',
    )
    return [normalize_mp_payment(p) for p in payments]

```

## Pipeline Playwright headful (semi-asistido)

Cadencia: ejecución semanal o on-demand vía CLI. Ricardo recibe un push del BE Pass / SMS al iniciar.

```

# adapters/scrapping_be.py – pseudocódigo

async def scrape_bancoestado(since: date) -> list[CanonicalTx]:
    state_path = vault.get_path('be_storage_state')
    async with async_playwright() as pw:
        browser = await pw.chromium.launch(headless=False) # HEADFUL
        ctx = await browser.new_context(
            storage_state=state_path if exists() else None
        )
        page = await ctx.new_page()
        await page.goto('https://www.bancoestado.cl')
        if not await is_logged_in(page):
            console.print('Por favor completa el login y aprueba BE Pass.')
            await page.wait_for_url('**/dashboard**', timeout=180_000)
            await ctx.storage_state(path=state_path)
        movs = await navigate_and_extract_movements(page, since)
        return [normalize_be_movement(m) for m in movs]

```

## 2.8 Motor ETL: normalización + categorización

### Pipeline ETL

Cada CanonicalTx que entra a staging pasa por las siguientes etapas. El orden es crítico (la categorización depende del merchant\_norm; el FITID depende del merchant\_norm).

1. Persistir crudo en raw\_artifacts (sha256 deduplica el artefacto en sí).
2. Normalizar fechas (todas a YYYY-MM-DD ISO).
3. Normalizar monto (Decimal con 2 decimales, signo absoluto, dirección por separado).
4. Normalizar merchant: aplicar reglas substr/regex de category\_rules ordenadas por prioridad.
5. Si ninguna regla matchea: aplicar fuzzy contra merchant\_norm históricos (RapidFuzz, threshold=85).
6. Si fuzzy tampoco: marcar needs\_review=1 y dejar merchant\_raw como merchant\_norm.
7. Calcular  $\text{fitid\_synthetic} = \text{sha256}(\text{owner}|\text{account\_alias}|\text{posted\_date}|\text{amount\_str}|\text{merchant\_norm})$ .
8. Detectar transferencia interna (ver 2.9): si encuentra par, marcar tx\_type='internal\_transfer' y enlazar.
9. Insertar / upsert en canonical\_tx por fitid\_synthetic. Si ya existe, fusionar campos preferiendo extracto > email > LLM > OCR.
10. Validar conciliación de balance (ver 2.10) — solo aplica para extractos completos, no para emails sueltos.

### Diccionario inicial de normalización

Se siembra al inicio con reglas de los comercios chilenos más frecuentes en sus emails:

| Pattern       | Type   | Merchant Norm          | Categoría          |
|---------------|--------|------------------------|--------------------|
| AMZN MKTP*    | substr | Amazon                 | Compras Online     |
| UBER *EATS    | substr | Uber Eats              | Comida a Domicilio |
| UBER *TRIP    | substr | Uber                   | Transporte         |
| PAYPAL *      | regex  | (según campo merchant) | (deferred)         |
| LIDER         | fuzzy  | Walmart Lider          | Supermercado       |
| JUMBO         | fuzzy  | Jumbo                  | Supermercado       |
| COPEC         | substr | Copac                  | Combustible        |
| SHELL         | substr | Shell                  | Combustible        |
| NETFLIX       | substr | Netflix                | Suscripciones      |
| ENTEL         | substr | Entel                  | Telefonía          |
| AGUAS ANDINAS | substr | Aguas Andinas          | Servicios Básicos  |
| ENEL          | substr | Enel                   | Servicios Básicos  |



## LLM fallback (Ollama + Qwen3 8B)

Cuando el regex de un parser específico falla (banco cambió plantilla), se invoca el LLM local con un prompt restrictivo:

```
# etl/llm_fallback.py

PROMPT = '''Extrae los siguientes campos del email bancario.
Responde SOLO con JSON válido, sin texto adicional, sin markdown.
Si un campo no se puede determinar, usa null.

Schema requerido:
{
  "posted_date": "YYYY-MM-DD",
  "amount": <number positivo, sin signo>,
  "direction": "debit"|"credit",
  "merchant_raw": "<string>",
  "card_last4": "<4 digits>"|null,
  "tx_type": "purchase"|"bill_payment"|"transfer_in"|"transfer_out"
}

Email a procesar:
<<<EMAIL>>>
{email_text}
<<<END>>>
'''

def parse_with_llm(email_text: str, hint: str | None) -> dict:
    response = ollama.chat(
        model='qwen3:8b',
        format='json',
        messages=[{'role': 'user',
                    'content': PROMPT.format(email_text=email_text)}],
        options={'temperature': 0.0},
    )
    data = json.loads(response['message']['content'])
    validate_schema(data)
    return data
```

Si el LLM también falla o devuelve JSON inválido, la transacción se marca `needs_review=1` con `review_reason='llm_fallback_failed'` y aparece en el reporte HTML.

## 2.9 Estrategia de deduplicación

### Cómo se calcula `fitid_synthetic`

```
# etl/fitid.py

def synthetic_fitid(tx: CanonicalTx) -> str:
    # Importante: usar merchant_NORM (ya canonicalizado), no raw
    # Importante: amount con dos decimales fijos, sin notación exponencial
    payload = '|'.join([
        tx.owner,
```

```

    tx.account_alias,
    tx.posted_date.isoformat() if tx.posted_date else 'NA',
    f'{tx.amount:.2f}',
    tx.direction,
    (tx.merchant_norm or tx.merchant_raw or 'UNKNOWN').upper(),
])
return hashlib.sha256(payload.encode('utf-8')).hexdigest()

```

### Sliding window para conciliación email ↔ extracto

Cuando un email llega el día 1 (autorización) y el extracto liquida el día 3 con el descriptor real, los hashes pueden divergir si merchant\_norm difiere ligeramente. La función match\_with\_sliding\_window agrupa transacciones candidatas:

```

def match_sliding_window(
    new_tx: CanonicalTx,
    window_days: int = 14,
) -> CanonicalTx | None:
    candidates = staging.find(
        owner=new_tx.owner,
        account_alias=new_tx.account_alias,
        amount=new_tx.amount,
        direction=new_tx.direction,
        date_from=new_tx.posted_date - timedelta(days=window_days),
        date_to =new_tx.posted_date + timedelta(days=window_days),
    )
    for c in candidates:
        sim = rapidfuzz.fuzz.token_set_ratio(
            c.merchant_norm or c.merchant_raw,
            new_tx.merchant_norm or new_tx.merchant_raw,
        )
        if sim >= 75:
            return c
    return None

```

### Reglas de fusión cuando hay match

- **Banco/extracto vence a email** en posted\_date, monto final exacto, referencia formal.
- **Email vence a banco** en texto descriptivo legible del comercio (los emails suelen tener nombres más limpios).
- **OCR/PDF rellena huecos** pero no derrota a OFX/CSV salvo que no haya otra fuente.
- **Si hay discrepancia en monto o dirección** no se fusionan: ambos se marcan needs\_review=1.

### Detección de transferencias internas

```

def detect_internal_transfer(
    tx: CanonicalTx,
    window_hours: int = 24,
) -> CanonicalTx | None:
    """Busca el contraparte de una transferencia interna.
    Ej: retiro $50.000 cuenta Ricardo → depósito $50.000 cuenta Laura.
    """
    if tx.tx_type not in ('transfer_in', 'transfer_out'):

```

```

        return None
    opposite = 'credit' if tx.direction == 'debit' else 'debit'
    candidate = staging.find_one(
        owner_in=['ricardo','laura'],          # cuentas familiares
        amount=tx.amount,
        direction=opposite,
        date_from=tx.posted_date - timedelta(hours=window_hours),
        date_to =tx.posted_date + timedelta(hours=window_hours),
        account_alias_neq=tx.account_alias,    # cuenta DISTINTA
    )
    if candidate is None:
        return None
    # Marcar ambos como par
    return candidate

```

## 2.10 Conciliación matemática ("Regla de Oro")

Tomado del informe de Gemini: cada extracto bancario procesado debe satisfacer:

$$B_{\text{final}} = B_{\text{initial}} + \sum \text{creditos} - \sum \text{debitos}$$

# Implementación:

```

def reconcile_account(account_alias, period_start, period_end):
    bal_ini = bank_reported_balance(account_alias, period_start)
    bal_fin = bank_reported_balance(account_alias, period_end)
    sum_c = sum(tx.amount for tx in txs if tx.direction == 'credit')
    sum_d = sum(tx.amount for tx in txs if tx.direction == 'debit')
    expected = bal_ini + sum_c - sum_d
    delta = bal_fin - expected
    if abs(delta) <= Decimal('0.01'):
        return ReconcileResult.OK
    elif abs(delta) <= bal_ini * Decimal('0.001'):
        return ReconcileResult.MINOR_ROUNDING
    else:
        return ReconcileResult.OFF          # bloquea writer

```

Si el resultado es OFF, ninguna transacción del lote se inyecta a finanza.mmb. El reporte HTML muestra delta y solicita revisión manual.

## 2.11 Estrategia multi-usuario

### Cuentas separadas (no datos separados)

Tomado del informe de NotebookLM: NO se crean dos archivos .mmb, ni dos bases de datos distintas. Se modela TODO en un único finanza.mmb con ACCOUNTID separados:

- BE\_Ricardo\_RUT (cuenta corriente Ricardo)
- BE\_Laura\_RUT (cuenta corriente Laura)
- CMR\_Ricardo (tarjeta CMR Ricardo)
- CMR\_Laura (tarjeta CMR Laura)
- MachBCI\_Ricardo, MP\_Ricardo, etc.

Esto permite ver el estado financiero familiar consolidado en MMEX y filtrar por cuenta cuando se necesita reporte individual.

## 2.12 Gastos compartidos: Tags

La tabla TAG\_V1 + TAGLINK\_V1 de MMEX permite etiquetar transacciones con múltiples tags. La estrategia es:

- **Tag 'Conjunto'** — gastos del hogar (supermercado, servicios básicos, vacaciones).
- **Tag 'Personal-R' / 'Personal-L'** — gastos individuales no compartidos.
- **Tag 'Reembolsable'** — gastos que un cónyuge debe a otro.

Las reglas de auto-tagging viven en category\_rules.tags\_json: por ejemplo, todo gasto en "Jumbo" se taggea automáticamente como Conjunto. Los reportes de MMEX permiten filtrar por (TagA OR TagB) y AND complejos.

## 2.13 Estructura de archivos del proyecto

```
C:\Finanzas\
├── finanza.mmb                # Money Manager Ex (NUNCA tocado salvo por writer)
├── backups\
│   ├── finanza_2026-05-02_pre.mmb    # snapshot inmediato pre-escritura
│   ├── finanza_2026-05-02_post.mmb
│   └── ... (rotación 30 días)
├── staging\
│   └── staging.db                # SQLite canónico
├── drop\                      # carpeta a vigilar (Watcher)
│   ├── ofx\
│   ├── csv\
│   └── pdf\
├── logs\
│   ├── 2026-05-02.log
│   └── ...
├── reports\                  # HTML diarios
│   └── review_2026-05-02.html
├── code\                    # repositorio git
│   ├── pyproject.toml
│   ├── README.md
│   ├── SPEC.md
│   ├── ARCHITECTURE.md
│   └── src\finanzasmex\
│       ├── __init__.py
│       ├── cli.py
│       ├── adapters\
│       ├── staging\
│       ├── etl\
│       ├── writer\
│       ├── orchestrator\
│       ├── ui\
│       └── secrets\
└── tests\
```

## 2.14 Concurrencia: el problema de MMEX abierto

SQLite gestiona bloqueos a nivel de archivo. Si MMEX está abierto en modo edición, finanza.mmb tiene un lock activo y un INSERT de Python recibirá database is locked.

Estrategia (informe Claude + Gemini):

11. Antes de cada escritura, el writer hace un SELECT 1 con timeout 0. Si recibe lock, marca el job como deferred.
12. Si está en modo schedule, simplemente reprograma para 1 hora después.
13. Si está en modo CLI manual, imprime un mensaje claro y sale con código 75 (EX\_TEMPFAIL).
14. Para Fase 1 (export OFX), el problema desaparece: Ricardo abre MMEX cuando él decide y le hace File → Import OFX.

### Por qué MMEX no soporta múltiples escritores simultáneos

MMEX en modo edición mantiene transacciones largas en SQLite que bloquean. SQLite con WAL permite "muchos lectores + un escritor" pero MMEX no usa WAL por default y mantiene escritores activos en sesiones largas. La solución correcta no es forzar concurrencia: es serializar. Si el writer encuentra lock, espera. Los datos siguen sanos en staging.db.

### 3. Decisiones técnicas (con justificación)

Esta sección documenta cada decisión donde los cuatro informes divergían y explica qué se eligió y por qué. Es deliberadamente más larga que las anteriores porque cada decisión queda registrada para no revisarla en plena implementación.

#### 3.1 Python local + SQLite directo (vs CSV en MVP)

- **Posiciones:** Gemini y NotebookLM proponen escritura SQL directa desde Fase 1. OpenAI (y Claude) recomiendan empezar con CSV/OFX y migrar a SQL en Fase 2.
- **Decisión:** Empezar con OFX export en Fase 1, migrar a SQL directo en Fase 2.
- **Razón:** El campo FITID YA EXISTE en CHECKINGACCOUNT\_V1 desde la utilidad OfxToMmex. Si exportamos OFX con FITID sintético, MMEX hace deduplicación nativa por nosotros — sin escribir una línea de código de SQL en MMEX. Esto valida toda la lógica de extracción y deduplicación antes de tocar el archivo .mmb directamente. Reduce el riesgo en órdenes de magnitud.

#### 3.2 OAuth Gmail API en lugar de IMAP

- **Posiciones:** OpenAI sugiere IMAP+local cache. Claude (y memoria del proyecto previo) recomienda Gmail API.
- **Decisión:** Gmail API.
- **Razón:** El proyecto previo de Ricardo ya implementó Gmail API con OAuth y refresh token persistente. IMAP requiere App Passwords (que Google está depreciando) o OAuth XOAUTH2 (que es más complejo que la API directa). Con Gmail API tenemos: (a) consultas server-side eficientes con label:Bancos, (b) marcado de mensajes como leídos opcional, (c) pagination nativa, (d) refresh token estable.

#### 3.3 Ollama/Qwen3 8B como fallback semántico

- **Posiciones:** NotebookLM enfatiza LLM local como capa primaria. Gemini y Claude lo consideran secundario.
- **Decisión:** Regex como capa primaria, LLM solo como fallback cuando el regex falla.
- **Razón:** Regex es 1000x más rápido y determinista cuando funciona. El LLM se invoca quizás en el 1–5% de los emails (cuando un banco cambia plantilla). El hardware del laptop (16GB RAM) corre Qwen3 8B en CPU con latencia ~10s, aceptable para esos casos puntuales pero NO para procesar 50 emails diarios. Además, force JSON-mode con format='json' garantiza output parseable.

#### 3.4 Playwright headful para BancoEstado y Falabella

- **Posiciones:** Todos los informes coinciden, pero Gemini sugería tantear headless primero.
- **Decisión:** Solo headful, semi-asistido. Ricardo presente al inicio.

- **Razón:** Información de campo (informe Claude): TLS fingerprinting de BancoEstado bloquea Chromium headless. BE Pass requiere humano + dispositivo móvil registrado. No vale la pena luchar contra esto: 5 minutos semanales de Ricardo es un costo aceptable. El storage\_state es válido por horas tras login, así que durante la sesión semi-auto se puede ejecutar todo el resto.

### 3.5 Mercado Pago vía OAuth

- **Decisión:** Authorization Code flow, una vez. Refresh automático.
- **Razón:** Es la única fuente con API legítima del set. No se debate. Implementación trivial con la lib oficial mercadopago de PyPI.

### 3.6 MachBCI: solo email + manual

- **Decisión:** No intentar scraping de la app móvil ni reverse engineering.
- **Razón:** Reverse engineering de la API privada (mitmproxy + emulador Android) viola los T&C de BCI y queda bajo Ley 21.459 art. 2 (acceso ilícito al sistema, no a los datos propios — la diferencia importa). Se deja como manual + email.

### 3.7 FITID sintético en CUSTOMFIELDDATA\_V1 (no en NOTES)

- **Posiciones:** Gemini propone NOTES (más simple). NotebookLM propone CUSTOMFIELDDATA\_V1. OpenAI propone TAG\_V1 o custom field.
- **Decisión:** CUSTOMFIELDDATA\_V1, en un campo personalizado dedicado llamado "sync\_hash".
- **Razón:** (a) NOTES es para que Ricardo escriba notas humanas; meter hash ahí lo ensucia. (b) Tags están reservados para semántica de gastos (Conjunto/Personal). (c) CUSTOMFIELDDATA\_V1 es la abstracción nativa de MMEX para metadatos extensibles, indexable, no contamina la UI principal.

### 3.8 Tags vs Categorías para gastos conjuntos

- **Decisión:** Tags (TAG\_V1).
- **Razón:** Categorías son taxonomía del TIPO de gasto (Comida, Transporte). Tags son metadata ortogonal (quién paga, si es reembolsable). Forzar gastos conjuntos en categorías obligaría a duplicar todas las categorías ("Comida-Conjunto", "Comida-Personal"). Tags + reportes con filtros (TagA | TagB) son más limpios y nativos en MMEX.

### 3.9 Backups: cómo y cuándo

- **Decisión:** Snapshot pre-escritura + post-escritura cada vez que el writer toca finanza.mmb. Rotación 30 días. Adicionalmente, Restic respaldando C:\Finanzas\ a un disco USB cifrado semanal.
- **Razón:** MMEX recomienda backups frecuentes. La doble copia (pre/post) permite diferenciar si un problema lo causó el writer o existe previamente. Restic agrega protección contra ransomware con cifrado y retención.

### 3.10 Cifrado: .mmb vs .emb vs contenedor BitLocker

- **Decisión:** Operación diaria sobre .mmb plano dentro de carpeta protegida por BitLocker. Conversión opcional a .emb solo si Ricardo decide después.
- **Razón:** (a) .emb usa wxSQLite3 con AES-256, NO compatible con sqlite3 estándar de Python sin recompilar. (b) BitLocker protege en reposo el laptop completo. (c) Si el threat model de Ricardo incluye "alguien con acceso físico al laptop encendido", entonces sí conviene .emb — pero requiere un módulo extra de descifrado/recifrado al inicio/fin de cada job. Decisión: Fase 1 plain .mmb + BitLocker; .emb queda en backlog.

### 3.11 Refactoring del proyecto previo gmail-to-MMEX

- **Contexto:** Ricardo ya tiene un proyecto spec-driven completado en Claude Code que sincroniza emails BancoEstado/Falabella a finanza.mmb.
- **Decisión:** FinanzasMMEX absorbe ese proyecto como un módulo (adapters/be\_email.py + adapters/cmr\_email.py). El proyecto anterior pasa a ser una sub-carpeta integrada, no un sistema rival.
- **Razón:** Reusar tests, regex y experiencia. El gmail-to-MMEX previo es la versión 0.1; FinanzasMMEX 1.0 es su evolución arquitectónica con multi-fuente, multi-usuario y dos capas.



## 4. Roadmap por fases

Cada fase entrega valor independiente. Si Ricardo se queda sin tiempo después de Fase 1, el sistema ya es útil. Las fases siguientes son refinamientos.

### Fase 0 — Setup (1–2 días)

| ID   | Tarea                                                                                        | Definition of Done                |
|------|----------------------------------------------------------------------------------------------|-----------------------------------|
| F0-1 | Crear repo git con estructura de carpetas. pyproject.toml con uv como package manager.       | structure committed               |
| F0-2 | Setup pre-commit (ruff, black, mypy strict).                                                 | ruff check pasa                   |
| F0-3 | Crear staging.db con schema.sql inicial. Wrapper repo.py.                                    | tests/test_schema.py pasa         |
| F0-4 | Implementar vault.py con keyring (Windows Credential Manager).                               | tests/test_vault.py pasa          |
| F0-5 | Setup OAuth Gmail (descargar credentials.json desde Google Cloud Console; primer authorize). | refresh_token guardado en keyring |
| F0-6 | Setup OAuth Mercado Pago (crear app en Developer Dashboard).                                 | refresh_token guardado            |
| F0-7 | Instalar Ollama localmente, descargar qwen3:8b. Test de inferencia.                          | ollama run qwen3:8b responde      |

### Fase 1 — MVP (3–5 días)

Meta: ingestar Gmail + Mercado Pago, exportar OFX, importar manualmente a MMEX. Sin tocar finanza.mmb desde Python aún.

| ID   | Tarea                                                                                                                                                                                    | Definition of Done                       |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| F1-1 | Implementar adapters/be_email.py con regex sobre 5 plantillas reales de BancoEstado (alertas de compra, transferencia recibida, transferencia enviada, pago de servicio, retiro cajero). | ≥ 95% extracción en suite de fixtures    |
| F1-2 | Implementar adapters/cmr_email.py para Falabella/CMR.                                                                                                                                    | Suite de fixtures pasa                   |
| F1-3 | Implementar adapters/mach_email.py (subset disponible).                                                                                                                                  | Funcional sobre las plantillas conocidas |
| F1-4 | Implementar adapters/mp_api.py contra API oficial de MP.                                                                                                                                 | Recupera 30 días con OAuth               |

| ID    | Tarea                                                                                         | Definition of Done                           |
|-------|-----------------------------------------------------------------------------------------------|----------------------------------------------|
| F1-5  | Implementar etl/normalize.py + etl/fuzzy.py + etl/fitid.py.                                   | Test de idempotencia pasa                    |
| F1-6  | Implementar etl/transfers.py (detección de internas).                                         | Test con caso Ricardo→Laura pasa             |
| F1-7  | Implementar etl/llm_fallback.py con Ollama.                                                   | Funciona offline; JSON validado              |
| F1-8  | Implementar writer/ofx_export.py: genera un archivo .ofx por cuenta a partir de canonical_tx. | MMEX lo importa sin errores                  |
| F1-9  | Implementar orchestrator/jobs.py con job_gmail y job_mp.                                      | CLI: finanzasmmx run --source gmail funciona |
| F1-10 | Implementar reporte HTML diario.                                                              | Se genera en reports/                        |
| F1-11 | Configurar Task Scheduler con dos tareas (gmail cada 1h, mp cada 6h).                         | Tareas activas; logs presentes               |
| F1-12 | Manual de usuario: cómo importar el OFX a MMEX.                                               | Documento en docs/import_guide.md            |

## Fase 2 — Writer SQLite directo (3–4 días)

Meta: el sistema ya escribe directamente sobre finanza.mmb sin intervención humana. Ricardo abre MMEX y ve los registros.

| ID   | Tarea                                                                                                                                                                            | Definition of Done                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| F2-1 | Crear field personalizado "sync_hash" en MMEX (vía UI, una sola vez). Capturar FIELDID.                                                                                          | FIELDID conocido y guardado en config |
| F2-2 | Implementar writer/mmex_lock.py: detección de archivo bloqueado.                                                                                                                 | Test simulado pasa                    |
| F2-3 | Implementar writer/backup.py: snapshot pre/post.                                                                                                                                 | Backups en backups/                   |
| F2-4 | Implementar writer/mmex_sql.py: INSERT con BEGIN IMMEDIATE / COMMIT / ROLLBACK. Crea/resuelve PAYEE_V1, CATEGORY_V1, TAG_V1, TAGLINK_V1, CUSTOMFIELDDATA_V1, CHECKINGACCOUNT_V1. |                                       |
| F2-5 | Implementar dedupe pre-INSERT: query a CUSTOMFIELDDATA_V1 buscando sync_hash existente.                                                                                          | No se crean duplicados al re-correr   |
| F2-6 | Test E2E: limpiar finanza.mmb de prueba, correr pipeline 2 veces, verificar idempotencia.                                                                                        | Test pasa                             |

| ID   | Tarea                                                            | Definition of Done    |
|------|------------------------------------------------------------------|-----------------------|
| F2-7 | Modo dual: CLI flag <code>--writer=ofx sql</code> para alternar. | Ambos modos funcionan |

### Fase 3 — Scraping headful (4–6 días)

| ID   | Tarea                                                                                                  | Definition of Done          |
|------|--------------------------------------------------------------------------------------------------------|-----------------------------|
| F3-1 | Implementar <code>adapters/scraping_be.py</code> con Playwright headful + <code>storage_state</code> . | Login funciona, extrae movs |
| F3-2 | Implementar <code>adapters/scraping_cmr.py</code> .                                                    | Login funciona, extrae movs |
| F3-3 | Sliding window dedup contra emails ya ingestados.                                                      | Sin duplicados              |
| F3-4 | Conciliación matemática contra balance reportado por el portal.                                        | Reconcile_log popula        |
| F3-5 | Workflow semi-asistido: CLI prompt para login interactivo.                                             | UX clara para Ricardo       |

### Fase 4 — Extras (3–5 días)

| ID   | Tarea                                                                                                                       | Definition of Done                      |
|------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| F4-1 | Implementar <code>adapters/ofx_loader.py</code> , <code>csv_loader.py</code> , <code>pdf_loader.py</code> para drop folder. | Tests pasan                             |
| F4-2 | Implementar Watcher de drop folder (watchdog).                                                                              | Procesa archivos nuevos automáticamente |
| F4-3 | Implementar UI Qt: ventana de revisión + quick-add manual.                                                                  | Funcional, integra con staging.db       |
| F4-4 | Implementar dashboards básicos: resumen mensual por categoría/tag, top comercios.                                           | HTML estáticos                          |
| F4-5 | Integración con n8n personal para notificación push (Telegram) cuando hay <code>needs_review</code> .                       | Notificación funciona                   |

### Fase 5 — Onboarding Laura (variable)

| ID   | Tarea                                                             | Definition of Done |
|------|-------------------------------------------------------------------|--------------------|
| F5-1 | Documentar acuerdo de mandato y consentimiento informado. Firmar. | PDF firmado        |

| ID   | Tarea                                                                                    | Definition of Done          |
|------|------------------------------------------------------------------------------------------|-----------------------------|
| F5-2 | Crear cuentas BE_Laura, CMR_Laura, etc. en MMEX. Setup tags joint/personal.              | Cuentas creadas             |
| F5-3 | Activar parsers de email para Laura (su Gmail propio o filtrado en el Gmail compartido). | Funcional                   |
| F5-4 | Sesión inicial de scraping headful con Laura para storage_state de sus cuentas.          | Almacenado en vault         |
| F5-5 | Refinar reglas de auto-tagging conjunto/personal según uso real.                         | Reportes mensuales precisos |

## 5. Plan de pruebas

### 5.1 Estrategia general

Cada Success Criterion del SPEC tiene al menos un test. La cobertura objetivo es  $\geq 70\%$  en módulos de parsing/etl,  $\geq 90\%$  en writer.

### 5.2 Tests unitarios

- **adapters:** fixtures con emails reales anonimizados (RUTs, montos randomizados); cada parser debe extraer la transacción esperada.
- **etl/fitid:** el mismo input debe producir el mismo hash siempre. Inputs ligeramente distintos (mayúsculas, espacios) deben producir el mismo hash si el merchant\_norm es el mismo.
- **etl/fuzzy:** casos conocidos ("AMZN MKTP\*1234" → Amazon).
- **etl/transfers:** par de transacciones que deben matchearse y otro par que no.
- **writer/mmex\_sql:** INSERT, ROLLBACK en error, idempotencia.

### 5.3 Tests de integración

- **E2E con MMEX dummy:** se crea un finanza\_test.mmb vacío, se inyecta lote de 50 transacciones, se verifica conteo y balances.
- **E2E idempotencia:** el mismo job corrido dos veces deja el mismo estado en finanza\_test.mmb.
- **E2E lock:** abrir finanza\_test.mmb desde otro proceso (sqlite3 con un BEGIN IMMEDIATE), correr el writer, verificar que difiere correctamente.
- **E2E reconciliación:** lote con balance que no cuadra debe terminar en "manual\_review" y NO insertar nada.

### 5.4 Tests manuales (smoke tests)

15. Correr finanzasmmex run --source=gmail con bandeja real de 24h. Verificar: ¿se crearon transacciones esperadas? ¿hay needs\_review razonables?
16. Importar el OFX generado en MMEX real (no test). Verificar: ¿MMEX muestra el FITID en cada registro? ¿no hay duplicados con import previos?
17. Borrar accidentalmente una transacción en MMEX, correr el pipeline de nuevo, verificar: ¿se restaura?
18. Cerrar Gmail temporalmente (revocar OAuth), correr el job, verificar: ¿error claro? ¿no corrompe staging?

### 5.5 Tests de seguridad

- Verificar que no hay credenciales en el repo (git secrets / detect-secrets).
- Verificar que el storage\_state de Playwright contiene cookies cifradas en keyring, no en disco plano.
- Test de exfiltración: monitorear con Wireshark mientras corre el pipeline. Solo conexiones a googleapis.com y mercadopago.com.

## 6. Operación y mantenimiento

### 6.1 Logging

Stack: structlog → archivo rotatorio en logs/. Formato JSON Lines.

```
{"ts": "2026-05-02T10:23:45Z", "level": "info",
  "job": "gmail_sync", "msg": "transactions_processed",
  "count": 17, "duration_ms": 3204}
```

Cada log tiene job, msg, level, ts. Eso permite consumir vía Sentinel si Ricardo decide ingestar a su SIEM personal.

### 6.2 Cuándo intervenir manualmente

| Disparador                      | Cadencia esperada | Acción                                                                                                            |
|---------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------|
| needs_review en reporte HTML    | Diaria            | Abrir reports/review_YYYY-MM-DD.html, revisar las pendientes, editarlas en MMEX o agregar regla a category_rules. |
| Storage_state de BE/CMR expira  | ~Semanal          | Correr finanzasmmex login --source=be; completar BE Pass.                                                         |
| Gmail token revocado            | Excepcional       | Re-autenticar OAuth (vault.refresh_oauth('gmail')).                                                               |
| MMEX upgrade de schema          | Cuando upgrade    | Probar en finanza_test.mmb antes de producción.                                                                   |
| Reconciliación off > 0.1%       | Excepcional       | Investigar lote en reconcile_log; probable tx faltante o duplicada.                                               |
| LLM fallback se activa > 5%/día | Cuando ocurra     | Banco cambió plantilla. Actualizar regex en parser correspondiente.                                               |

### 6.3 Procedimiento de restauración

19. Cerrar MMEX si está abierto.
20. Renombrar finanza.mmb a finanza\_corrupted\_\$(date).mmb.
21. Copiar el último backup OK desde backups/finanza\_YYYY-MM-DD\_post.mmb a finanza.mmb.
22. En staging.db, marcar todas las transacciones inserted desde la fecha del backup como pending.
23. Correr finanzasmmex replay --since=YYYY-MM-DD para re-aplicar.
24. Verificar conteo y balance en MMEX.

### 6.4 Mantenimiento del diccionario de categorización

category\_rules es la base más "viva" del sistema. Se crece manualmente en sqlite browser cuando aparece un comercio nuevo. Convención:

- Reglas substr van primero (más rápidas y precisas).
- Reglas regex para casos con variación numérica ("AMZN MKTP\*\d+").
- Reglas fuzzy para errores tipográficos sistemáticos del banco.
- priority numérica menor = se evalúa antes.



## 7. Riesgos y mitigaciones

| ID  | Riesgo                                                                     | Probab.          | Impacto | Mitigación                                                                                                                                        |
|-----|----------------------------------------------------------------------------|------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| R1  | BancoEstado / Falabella detectan automatización y bloquean cuenta.         | Medio            | Alto    | Headful + cadencia humana semanal + storage_state de sesión humana. No paralelizar logins. Si pasa: usar export manual desde el portal hasta SFA. |
| R2  | Banco cambia plantilla de email; transacciones se pierden silenciosamente. | Alto             | Medio   | Alert en reporte HTML cuando LLM fallback > 5%. Tests con fixture-suite de plantillas. Monitor diario los primeros 30 días.                       |
| R3  | Corrupción de finanza.mmb durante INSERT.                                  | Bajo             | Crítico | Backup pre-escritura + ACID + lock detection + idempotencia. Capacidad de replay desde staging.                                                   |
| R4  | Pérdida de tokens OAuth por reset de Windows.                              | Bajo             | Medio   | keyring respaldado en password manager. Documentar procedimiento de re-autenticación.                                                             |
| R5  | Ollama deja de responder (CPU saturada por Defender, etc.).                | Medio            | Bajo    | LLM es fallback, no ruta crítica. Si falla, transacción va a needs_review y se procesa después.                                                   |
| R6  | Mercado Pago revoca app o cambia API.                                      | Bajo             | Bajo    | App propia sigue siendo de Ricardo; cambios de API requieren actualizar adapter, pero no afectan al resto.                                        |
| R7  | MachBCI nunca expone API a personas; se mantiene gap de cobertura.         | Alto             | Bajo    | Aceptado como restricción. Manual + emails de notificación cubren la mayoría.                                                                     |
| R8  | SFA llega antes de lo previsto y obliga a refactor grande.                 | Medio (positivo) | Medio   | Adapters pluggable. Migrar scraping_be.py → sfa_be.py es semanas de trabajo, no meses.                                                            |
| R9  | Laura revoca consentimiento.                                               | Bajo             | Medio   | Acuerdo firmado prevé revocación. Borrar storage_state, deshabilitar parsers de Laura, conservar histórico ya importado.                          |
| R10 | El proyecto pierde momentum; queda inconcluso.                             | Medio            | Medio   | Roadmap por fases. Fase 1 ya entrega valor (export OFX). Fases 2+ son mejora incremental.                                                         |
| R11 | Ricardo cambia de laptop o reinstala Windows.                              | Medio            | Bajo    | Repo en git remoto privado (GitHub o self-hosted). Tokens en keyring respaldado. Backup mensual a USB cifrado con Restic.                         |

| ID  | Riesgo                                             | Probab. | Impacto | Mitigación                                                                                                                                                |
|-----|----------------------------------------------------|---------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| R12 | Disputa con BancoEstado sobre acceso automatizado. | Bajo    | Medio   | Documentar acuerdo familiar, mantener cadencia humana, no paralelizar, no acceder a cuentas de terceros. La defensa es "acceso con credenciales propias". |

## Apéndice A — Origen de cada decisión por informe

Tabla de trazabilidad: qué tomó este plan de cada uno de los cuatro informes y qué se descartó.

| Decisión                                                        | Origen principal    | Por qué se mantuvo                                                                                             |
|-----------------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------|
| Arquitectura de dos capas (staging.db + finanza.mmb)            | OpenAI              | Defendida también por Gemini y NotebookLM con énfasis distinto. Es el corazón del plan.                        |
| FITID sintético en CUSTOMFIELDDATA_V1                           | NotebookLM + Gemini | Aprovecha el FITID nativo del schema v21 + el subsistema de campos personalizados. Decisión final consolidada. |
| Sliding window 14 días para conciliación email↔extracto         | Gemini              | Crítico para evitar que la misma compra entre dos veces.                                                       |
| Regla de oro de conciliación matemática                         | Gemini              | $B_{final} = B_{initial} + \sum \text{creditos} - \sum \text{debitos}$ . Validación obligatoria pre-write.     |
| Tags vs Categorías para gastos conjuntos                        | NotebookLM + OpenAI | Mejor práctica para multi-usuario sin duplicar taxonomía.                                                      |
| Mercado Pago como única API legítima del set                    | Claude              | Realismo de campo crítico. Los demás informes asumían APIs disponibles que NO existen.                         |
| Estado real de BancoEstado (TLS fingerprinting, BE Pass)        | Claude              | Información de campo que ningún otro informe tenía. Determina la decisión headful + semi-auto.                 |
| MachBCI sin API ni web personal                                 | Claude              | Restricción técnica que hay que aceptar; informes anteriores la asumían superable.                             |
| Marco legal Chile (21.459 / 21.521 SFA / 21.719)                | Claude              | Único informe con análisis legal específico chileno y cronograma SFA.                                          |
| LLM local Ollama/Qwen como fallback                             | NotebookLM          | Alineado con stack del laptop de Ricardo (Qwen3 8B en CPU).                                                    |
| Modelo canónico CanonicalTx con campos de origen y trazabilidad | OpenAI              | Esquema más rico que MMEX, con audit trail.                                                                    |
| Detección de transferencias internas                            | Gemini              | Crítica para no inflar artificialmente flujo de caja.                                                          |
| Custom fields como abstracción nativa MMEX para metadata        | NotebookLM + Gemini | Solución elegante que no contamina NOTES.                                                                      |
| Camelot para PDFs con bordes; pdfplumber para coordenadas       | Gemini + OpenAI     | Convergencia clara. Plan adopta ambos según tipo de PDF.                                                       |
| BeautifulSoup para HTML embedido en emails                      | Gemini + OpenAI     | Estándar del ecosistema.                                                                                       |

| Decisión                        | Origen principal | Por qué se mantuvo                                                                         |
|---------------------------------|------------------|--------------------------------------------------------------------------------------------|
| Comparativa Fintoc/Floid vs DIY | Claude           | Informa la decisión: \$0 (DIY) vs $\geq$ \$260.000/mes (Fintoc). DIY gana por presupuesto. |

## Decisiones donde el plan diverge de los informes

| Decisión divergente                                | Informe original sugería | Por qué este plan elige distinto                                                                                                                                       |
|----------------------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Empezar con SQL directo a finanza.mmb desde Fase 1 | Gemini + NotebookLM      | El plan empieza con OFX export. Razón: el riesgo de corrupción es muy alto y MMEX ya hace dedupe nativa por FITID. SQL directo entra en Fase 2 sobre base ya validada. |
| IMAP en lugar de Gmail API                         | OpenAI                   | Gmail API es más estable, ya implementada en proyecto previo. App Passwords se están depreciando.                                                                      |
| Syncthing para multi-dispositivo desde el inicio   | NotebookLM               | Diferido a fase futura. Resuelve un problema que Ricardo no tiene aún (un solo laptop, MMEX en local).                                                                 |
| LLM como capa primaria                             | NotebookLM               | Costo computacional no se justifica. Regex es 1000x más rápido cuando funciona; LLM solo como fallback.                                                                |
| Reverse engineering de MachBCI                     | Gemini implícitamente    | Viola T&C BCI y queda bajo Ley 21.459 art. 2. Inaceptable.                                                                                                             |

## Apéndice B — Esquema SQL completo de staging.db

Versión inicial. Las migraciones futuras viven en `src/finanzasmmex/staging/migrations/`.

```
-- staging.db schema v1
-- 2026-05-02

PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

CREATE TABLE schema_version (
  version INTEGER PRIMARY KEY,
  applied_at TEXT NOT NULL DEFAULT (datetime('now'))
);
INSERT INTO schema_version (version) VALUES (1);

-- ===== canonical_tx =====
CREATE TABLE canonical_tx (
  tx_uid          TEXT PRIMARY KEY,
  owner           TEXT NOT NULL CHECK (owner IN ('ricardo','laura','joint')),
  source_type     TEXT NOT NULL,
  source_file     TEXT,
  source_ref      TEXT,
  content_sha256  TEXT NOT NULL,
  raw_text        TEXT,
  event_date     TEXT,
  booking_date    TEXT,
  posted_date     TEXT,
  amount          REAL NOT NULL CHECK (amount > 0),
  currency        TEXT NOT NULL DEFAULT 'CLP',
  direction       TEXT NOT NULL CHECK (direction IN ('debit','credit')),
  account_alias   TEXT NOT NULL,
  card_last4      TEXT,
  merchant_raw    TEXT,
  merchant_norm   TEXT,
  tx_type         TEXT NOT NULL,
  category_guess  TEXT,
  subcategory_guess TEXT,
  tags_json       TEXT NOT NULL DEFAULT '[]',
  fitid_synthetic TEXT NOT NULL,
  parser_name     TEXT NOT NULL,
  parser_version  TEXT NOT NULL,
  needs_review    INTEGER NOT NULL DEFAULT 0,
  review_reason   TEXT,
  mmex_account_id INTEGER,
  mmex_tx_id      INTEGER,
  mmex_status     TEXT NOT NULL DEFAULT 'pending',
  transfer_pair_uid TEXT,
  created_at      TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at      TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (transfer_pair_uid) REFERENCES canonical_tx(tx_uid)
);

CREATE UNIQUE INDEX uq_canonical_fitid ON canonical_tx(fitid_synthetic);
```

```

CREATE      INDEX idx_canonical_owner_date ON canonical_tx(owner, posted_date);
CREATE      INDEX idx_canonical_status      ON canonical_tx(mmex_status);
CREATE      INDEX idx_canonical_match
      ON canonical_tx(owner, account_alias, amount, posted_date, merchant_norm);

-- ===== raw_artifacts =====
CREATE TABLE raw_artifacts (
  artifact_id      TEXT PRIMARY KEY,
  artifact_type    TEXT NOT NULL,
  source_ref       TEXT NOT NULL,
  content_sha256   TEXT NOT NULL,
  payload_blob     BLOB,
  payload_text     TEXT,
  fetched_at       TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE UNIQUE INDEX uq_raw_sha ON raw_artifacts(content_sha256);

-- ===== category_rules =====
CREATE TABLE category_rules (
  rule_id          INTEGER PRIMARY KEY,
  pattern           TEXT NOT NULL,
  pattern_type      TEXT NOT NULL CHECK (pattern_type IN ('substr','regex','fuzzy')),
  merchant_norm     TEXT NOT NULL,
  payee_id_mmex    INTEGER,
  category_name     TEXT NOT NULL,
  subcategory_name  TEXT,
  tags_json         TEXT NOT NULL DEFAULT '[]',
  fuzzy_threshold   INTEGER DEFAULT 85,
  priority          INTEGER NOT NULL DEFAULT 100,
  active            INTEGER NOT NULL DEFAULT 1,
  created_at        TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_rules_active_priority ON category_rules(active, priority);

-- ===== merge_log =====
CREATE TABLE merge_log (
  merge_id         INTEGER PRIMARY KEY,
  fitid_synthetic  TEXT NOT NULL,
  sources_json     TEXT NOT NULL,
  chosen_source    TEXT NOT NULL,
  merged_at        TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_merge_fitid ON merge_log(fitid_synthetic);

-- ===== reconcile_log =====
CREATE TABLE reconcile_log (
  reconcile_id     INTEGER PRIMARY KEY,
  account_alias    TEXT NOT NULL,
  period_start     TEXT NOT NULL,
  period_end       TEXT NOT NULL,
  balance_initial  REAL NOT NULL,
  balance_final    REAL NOT NULL,
  sum_credits      REAL NOT NULL,
  sum_debits       REAL NOT NULL,
  expected_final   REAL NOT NULL,

```

```

    delta            REAL NOT NULL,
    status           TEXT NOT NULL CHECK (status IN ('ok','minor','off','manual_review')),
    notes            TEXT,
    created_at       TEXT NOT NULL DEFAULT (datetime('now'))
);

-- ===== job_runs =====
CREATE TABLE job_runs (
    run_id           TEXT PRIMARY KEY,
    job_name         TEXT NOT NULL,
    started_at       TEXT NOT NULL,
    finished_at      TEXT,
    status           TEXT NOT NULL CHECK (status IN ('running','ok','error','deferred')),
    items_processed  INTEGER NOT NULL DEFAULT 0,
    items_inserted   INTEGER NOT NULL DEFAULT 0,
    items_review     INTEGER NOT NULL DEFAULT 0,
    error_message    TEXT,
    metadata_json    TEXT
);
CREATE INDEX idx_job_runs_started ON job_runs(started_at DESC);

```

## Apéndice C — Patrones regex iniciales (BancoEstado)

Patrones de Python regex (re module) para los 5 tipos de notificación más frecuentes de BancoEstado. Sirven como punto de partida; se ajustarán contra fixtures reales en F1-1.

### Compra con tarjeta de crédito

```
PATTERN_BE_TC_PURCHASE = re.compile(
    r'compra\s+por\s+\$\s*(?P<amount>[\d.]+)'
    r'.*?en\s+(?P<merchant>.+)\s+con',
    re.IGNORECASE | re.DOTALL
)

PATTERN_BE_TC_DATE = re.compile(
    r'(?P<date>\d{1,2}/\d{1,2}/\d{4})\s+\d{2}:\d{2}',
)

PATTERN_BE_TC_CARD = re.compile(
    r'tarjeta\s+terminada\s+en\s+(?P<last4>\d{4})',
    re.IGNORECASE
)
```

### Transferencia recibida

```
PATTERN_BE_TRANSFER_IN = re.compile(
    r'recibiste\s+\$\s*(?P<amount>[\d.]+)'
    r'.*?desde\s+(?P<from_name>.+)\s+(?:RUT|e1)',
    re.IGNORECASE | re.DOTALL
)
```

### Transferencia enviada

```
PATTERN_BE_TRANSFER_OUT = re.compile(
    r'transferiste\s+\$\s*(?P<amount>[\d.]+)'
    r'.*?a\s+(?P<to_name>.+)\s+(?:\s+RUT|\s+e1|\.)',
    re.IGNORECASE | re.DOTALL
)
```

### Pago automático de servicio

```
PATTERN_BE_BILL = re.compile(
    r'pago\s+(?:de\s+cuenta|automatico)\s+'
    r'(?P<por>\s+)\$\s*(?P<amount>[\d.]+)'
    r'.*?(?:a|de)\s+(?P<service>.+)\s+(?:e1|fue)',
    re.IGNORECASE | re.DOTALL
)
```

### Retiro en cajero

```
PATTERN_BE_ATM = re.compile(
    r'retiro\s+(?:en\s+cajero\s+)?'
    r'por\s+\$\s*(?P<amount>[\d.]+)'
    r'.*?(?:cajero|ATM)\s+(?P<location>.+)\s+(?:\s+e1|\.)',
    re.IGNORECASE | re.DOTALL
)
```



## Helper de normalización de montos chilenos

```
def parse_clp_amount(raw: str) -> Decimal:
    """\$.234.567 o \$.234.567,89 → Decimal."""
    cleaned = raw.replace('\$', '').replace(' ', '').strip()
    if ',' in cleaned:
        # formato chileno: punto miles, coma decimales
        cleaned = cleaned.replace('.', '').replace(',', '.')
    else:
        # solo punto miles
        cleaned = cleaned.replace('.', '')
    return Decimal(cleaned)
```

## Apéndice D — Compliance chileno: detalle adicional

### D.1 Posición legal de la operación

La operación FinanzasMMEX se sostiene en cuatro principios legales explícitos:

25. Acceso con credenciales propias: el art. 2 de la Ley 21.459 castiga el acceso indebido. El acceso con credenciales propias y voluntarias no es indebido. La defensa jurídica está en la voluntariedad.
26. Consentimiento expreso de Laura: el acuerdo escrito (Apéndice E) documenta que ella entrega sus credenciales con conocimiento del uso.
27. Proporcionalidad y mínimo dato: el sistema solo lee movimientos. No realiza transacciones. No accede a productos de inversión. No comparte con terceros.
28. Cumplimiento Ley 21.719 (datos personales): titular del dato es el cónyuge. Mantenimiento local + cifrado en reposo + posibilidad de borrado por revocación = cumplimiento sustantivo.

### D.2 Riesgo contractual con bancos

Los T&C de BancoEstado y Banco Falabella prohíben "acceso automatizado al servicio". El riesgo no es penal sino contractual:

- Suspensión temporal de cuenta (caso más probable).
- Bloqueo de credenciales digitales con requerimiento de regeneración presencial.
- Pérdida de servicios digitales (BE Pass, transferencias online).

Mitigación operacional:

- Cadencia humana:  $\leq 1$  sesión scraping por banco por semana.
- Sin paralelismo: una cuenta a la vez.
- Sin user-agent custom ni headers spoofeados — Playwright headful con Chrome real.
- Sin operaciones transaccionales: solo lectura de movimientos.
- Si se detecta CAPTCHA, abortar sesión y notificar.

### D.3 Cronograma SFA (Sistema de Finanzas Abiertas)

La NCG entra en vigencia 4 julio 2026 con implementación gradual. Los primeros sujetos obligados son los grandes bancos. Cronograma estimado:

| Fecha estimada | Hito                                                                            |
|----------------|---------------------------------------------------------------------------------|
| jul 2026       | NCG vigente. Bancos grandes inician trabajo de implementación.                  |
| ene 2027       | Primeros endpoints de "datos abiertos" (productos básicos, sin consentimiento). |
| jul 2027       | Endpoints con consentimiento del usuario (movimientos, saldos).                 |

| Fecha estimada | Hito                                                                                     |
|----------------|------------------------------------------------------------------------------------------|
| ene 2028       | Bancos minoristas (BancoEstado, Falabella) cubiertos.                                    |
| jul 2028       | Cobertura completa — punto en que FinanzasMMEX podría migrar de scraping a API regulada. |

Implicancia arquitectónica: las fuentes deben ser pluggable. En 2027–2028 se reemplaza `scraping_be.py` por `sfa_be.py` sin tocar el resto del sistema. Esa pluggabilidad es la razón principal del modelo CanonicalTx + adapter pattern.

## Apéndice E — Plantilla de acuerdo de mandato

*Plantilla mínima para el acuerdo entre Ricardo y Laura. Una página, español neutro, archivada como PDF firmado.*

### ACUERDO DE MANDATO Y CONSENTIMIENTO INFORMADO PARA GESTIÓN FINANCIERA FAMILIAR

Entre Ricardo [apellidos], RUT [\_\_\_\_], en adelante "el Mandatario", y Laura [apellidos], RUT [\_\_\_\_], en adelante "la Mandante", se conviene lo siguiente:

**PRIMERO — Objeto.** La Mandante autoriza al Mandatario a acceder, exclusivamente para fines de consolidación financiera familiar, a los portales digitales y aplicaciones móviles de las siguientes instituciones donde la Mandante mantiene productos: Banco Estado de Chile, Banco Falabella / Tarjeta CMR, y cualquier otra que ambas partes acuerden por escrito en el futuro.

**SEGUNDO — Alcance.** El acceso se limita a operaciones de SOLO LECTURA: visualización de movimientos, saldos y estados de cuenta. Queda EXPRESAMENTE PROHIBIDO al Mandatario realizar transferencias, pagos, contrataciones de productos, modificaciones de claves, o cualquier acción transaccional.

**TERCERO — Tratamiento de datos.** La información obtenida se almacena exclusivamente en la PC personal del Mandatario, con cifrado en reposo. No se comparte con terceros, no se sube a servicios en la nube de proveedores externos, y se utiliza únicamente como insumo de la herramienta de gestión financiera familiar Money Manager Ex.

**CUARTO — Revocación.** La Mandante puede revocar este mandato en cualquier momento mediante comunicación escrita al Mandatario. La revocación obliga al Mandatario a: (a) cesar inmediatamente todo acceso, (b) eliminar las credenciales almacenadas, (c) borrar los datos derivados que la Mandante solicite por escrito.

**QUINTO — Vigencia.** Este acuerdo rige desde la fecha de firma y mientras subsista la voluntad de ambas partes.

Firmado en [ciudad], el [fecha].

---

Ricardo [apellidos]

RUT [\_\_\_\_]

---

Laura [apellidos]

RUT [\_\_\_\_]

**Disclaimer no legal**

Esta plantilla es un punto de partida documental, no un instrumento legal validado. Para uso formal en disputas, conviene revisarla con un abogado chileno familiarizado con la Ley 21.719 y la Ley 21.459. Para uso interno familiar (que es el caso aquí), tiene valor probatorio razonable como evidencia de consentimiento expreso.

## Cierre y siguientes pasos

Este documento es la "single source of truth" del proyecto FinanzasMMEX. Cualquier divergencia entre el código y este plan debe resolverse aquí (Phase 6 del spec-driven-workflow), no en la implementación.

### Próximos pasos sugeridos

29. Revisar este plan (Ricardo). Aprobar o pedir ajustes.
30. Iniciar Fase 0: setup de repo, OAuth Gmail, OAuth Mercado Pago, Ollama.
31. Comenzar Fase 1 con un solo banco (BancoEstado vía email) hasta tener un loop end-to-end funcionando: Gmail → staging.db → OFX → import a finanza.mmb.
32. Ejecutar 1 semana de uso real en "shadow mode" (export OFX, importación manual a un finanza\_test.mmb separado) antes de tocar el archivo productivo.
33. Una vez validado, migrar finanza.mmb productivo y activar Task Scheduler.

### Contacto y mantenimiento del documento

Este plan es un documento vivo. Cuando algo cambie sustancialmente (banco modifica plantilla, SFA llega antes, Laura entra antes de Fase 5), se actualiza la sección correspondiente y se incrementa la versión en la portada.

— Fin del plan —